LAB 3-3 FULL-STACK DATA INTEGRATION USING FLASK REST API AND REACT DASHBOARD

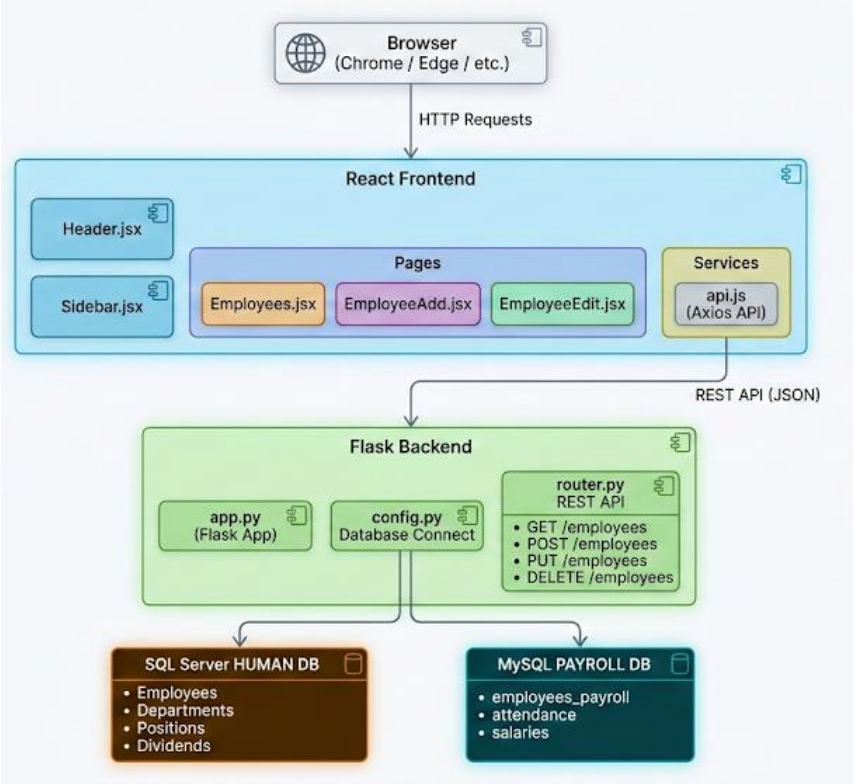
PHẦN 1 - MỤC TIÊU BÀI LAB:

Sau buổi thực hành này, sinh viên sẽ biết:

- *React là gì và vì sao dÙng*
- *Cài đặt môi trường React*
- *Tạo Project trên React*
- *Chạy React và Python Flask song song*
- *FE React gọi API từ Flask*
- *Tạo Dashboard FE React gọi API đầy đủ các chức năng như danh sách nhân viên, thêm mới hồ sơ nhân viên, cập nhật hồ sơ nhân viên, xóa hồ sơ nhân viên.*

PHẦN 2 - SYSTEM ARCHITECTURE

Sơ đồ kiến trúc của Component Diagram – React + Flask



PHẦN 3 – KIẾN THỨC CƠ BẢN

1. React là gì?

React là **thư viện JavaScript** dùng để xây dựng giao diện web:

- Dùng để làm **FE (Front-end)** – giao diện người dùng
- Tạo web dạng **SPA – Single Page Application**
- Giao diện chạy rất nhanh
- Gọi API từ Backend để lấy dữ liệu

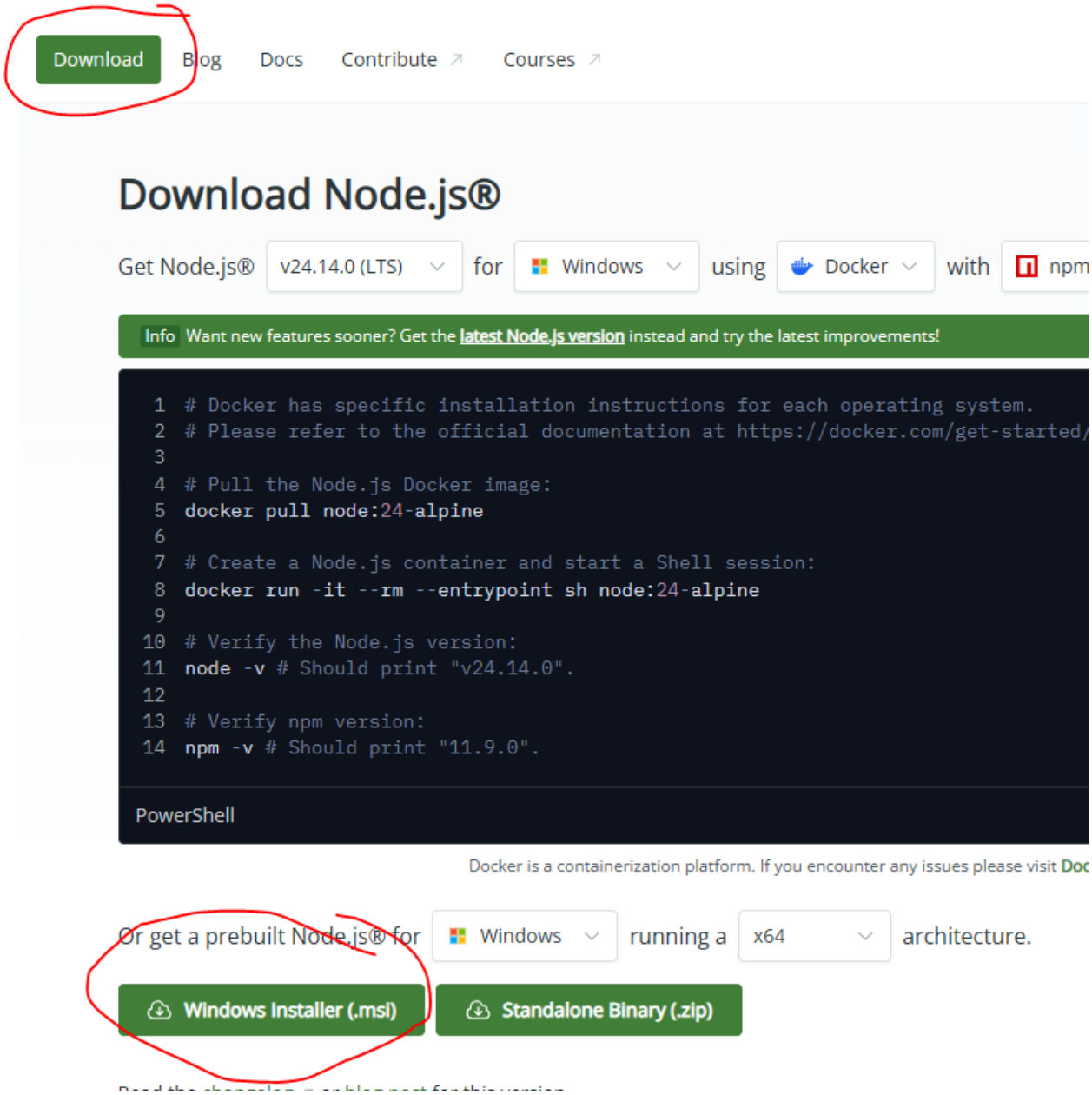
Trong dự án này:

- **Backend: Flask + Python** (các API sẽ được viết trong **router.py**)
- **Frontend: React**

PHẦN 4 – CÀI ĐẶT MÔI TRƯỜNG

2.1. Cài Node.js

React cần Node.js để chạy. Vào website: https://nodejs.org



Download nodejs về và tiến hành cài đặt, Sau khi cài xong vào **Command Prompt**, gõ lênh sau để kiểm tra:

node -v

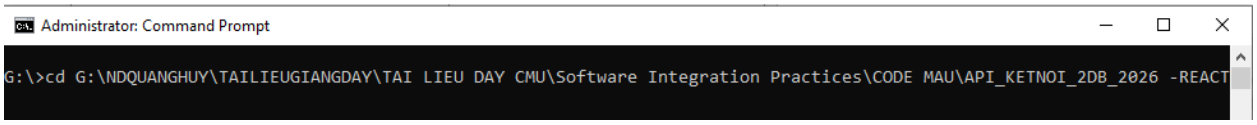
npm -v

Nếu ra số phiên bản là đã cài thành công

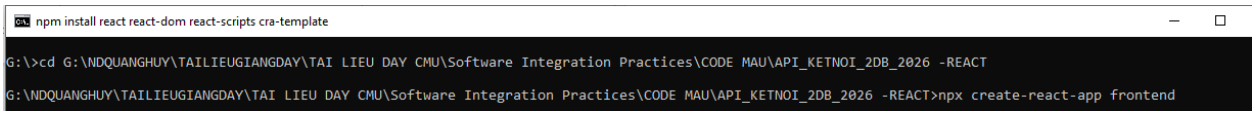
PHẦN 5 – TẠO PROJECT REACT

Tạo 01 thư mục project tên là **API_KETNOI_2DB_2026 -REACT**

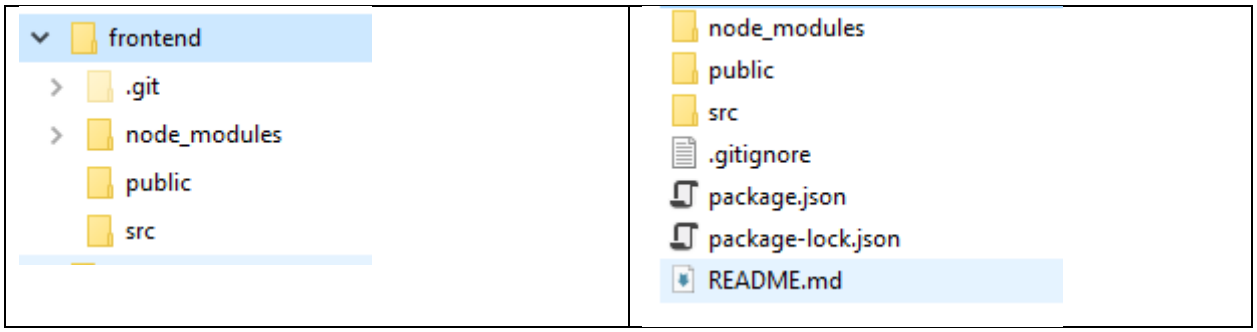
Mở Command Prompt ra và gõ lệnh CD đến chuyển đến thư mục chứa project



Sau đó, tạo Project React bằng lệnh sau trong Command prompt **npx create-react-app frontend**



Sau khi chạy xong, thư mục dự án sẽ có cấu trúc:



Giải thích:

Thư mục	Ý nghĩa
node_modules	**Đây là thư mục quan trọng KHÔNG ĐƯỢC CHỈNH SỬA gì trong thư mục này.** Thư mục này chứa toàn bộ **thư viện React** cần (*ví dụ: react, react-dom, bootstrap, axios,…*) *Lưu ý:* • **Không bao giờ mở file bên trong.** • **Không commit lên GitHub** (vì rất nặng). • Nếu bị lỗi chỉ cần xóa và chạy lại: **npm install**

<table>
<tr><td>public</td><td>Đây là nơi chứa các file tĩnh, được React dùng để hiển thị website.
Bên trong sẽ có:
<table>
<tr><th>File</th><th>Ý nghĩa</th></tr>
<tr><td>index.html</td><td>File HTML duy nhất React render App vào</td></tr>
<tr><td>favicon.ico</td><td>Icon website</td></tr>
<tr><td>manifest.json</td><td>Thông tin app (Progressive Web App)</td></tr>
<tr><td>robots.txt</td><td>Cấu hình cho SEO</td></tr>
</table>
<u>Lưu ý:</u>
<ul>
<li>React chỉ có 1 file HTML duy nhất public/index.html</li>
<li>App React sẽ được nhúng vào thẻ trong file này</li>
</ul></td></tr>
<tr><td>src/</td><td>Đây là thư viện quan trọng nhất
Là trái tim của ứng dụng React. Là nơi sẽ lập trình toàn bộ giao diện React.
Bên trong sẽ có:
<table>
<tr><th>File</th><th>Công dụng</th></tr>
<tr><td>index.js</td><td>File chạy React đầu tiên (khởi động app)</td></tr>
<tr><td>App.js</td><td>File chính điều khiển giao diện React</td></tr>
<tr><td>App.css</td><td>CSS chính</td></tr>
<tr><td>index.css</td><td>CSS chung</td></tr>
<tr><td>logo.svg</td><td>Logo React demo</td></tr>
</table></td></tr>
</table>

<table>
<tr><td></td><td>
<table>
<tr><td>(Folder chúng ta sẽ tạo)</td><td>pages/, components/, services/, …</td></tr>
</table>

Đây là nơi chúng ta sẽ làm việc nhiều nhất:
Tạo folder:
• src/pages/ : chứa các trang: Employees, Add, Edit
• src/components/ : menu, header
• src/services/ :nơi viết API call (axios)
</td></tr>
<tr><td>File
package.json</td><td>
Đây là “trái tim quản lý project”.
Công dụng của file này:
<table>
<tr><th>Mục</th><th>Ý nghĩa</th></tr>
<tr><td>"name"</td><td>tên project React</td></tr>
<tr><td>"dependencies"</td><td>các thư viện dùng cho React</td></tr>
<tr><td>"scripts"</td><td>các lệnh như npm start, npm build</td></tr>
<tr><td>"version"</td><td>phiên bản project</td></tr>
<tr><td>"devDependencies"</td><td>thư viện chỉ dùng khi phát triển</td></tr>
</table>
Lệnh quan trọng trong package.json:
Khi chúng ta chạy:
npm start
npm install axios
npm run build
Những lệnh này dựa vào thông tin trong package.json.
</td></tr>
</table>

PHẦN 6 – CẤU TRÚC PROJECT REACT CỦA LAB

frontend/

├─ src/

```
│   ├── pages/
│   │   ├── Employees.jsx
│   │   ├── EmployeeAdd.jsx
│   │   ├── EmployeeEdit.jsx
│   ├── components/
│   │   ├── Header.jsx
│   ├── App.js
│   ├── index.js
├── public/
│   └── index.html
├── package.json
└── node_modules/
```

PHẦN 7: THIẾT KẾ FRONT-END VỚI REACT

BƯỚC 1 — GIẢI THÍCH CHI TIẾT TỪNG FILE TRONG src/

Sau khi tạo React bằng lệnh **npx create-react-app frontend** (đã tạo trước đó), thì trong thư mục **frontend/src/** sẽ có:

```
src/
├── App.js
├── App.css
├── index.js
├── index.css
├── reportWebVitals.js
├── setupTests.js
```

```
├── logo.svg
```

Trong bài LAB này, chỉ cần quan tâm đến 2 file quan trọng nhất: **App.js** và **index.js, c**ác file còn lại có thể giữ nguyên hoặc xóa.

1. File index.js — File KHỞI ĐỘNG của React

Đây là file đầu tiên chạy khi khởi động React (npm start).

Chức năng:

- Gọi ReactDOM để hiển thị toàn bộ ứng dụng
- Render <App /> vào thẻ trong public/index.html

Chú ý: Sinh viên KHÔNG cần sửa file này.

2. File App.js — File QUAN TRỌNG NHẤT của cả dự án

- Vị trí: **frontend/src/App.js**
- Đây là **"bộ não giao diện React"**.
- Tất cả **layout, router, UI chính** đều được điều khiển từ đây.

3. File App.css — CSS cho App.js

- Đây là nơi chứa **CSS riêng cho App.js**
- Có thể xóa hoặc giữ tùy thích.

Trong dự án thực tế: Ta dùng **Bootstrap**, nên file này **ít dùng**.

4. File index.css — CSS chung toàn dự án

- File này chạy trước toàn bộ React.
- Có thể thêm các style toàn cục (font, màu nền).

  Ví dụ:

  ```
  body {
    background: #f5f5f5;
  }
  ```

5. File logo.svg, reportWebVitals.js, setupTests.js

Có thể **xoá**, vì:

- logo.svg: chỉ dùng demo
- reportWebVitals: dùng đo hiệu năng, sinh viên không cần
- setupTests: dùng cho test, không cần

Không ảnh hưởng React nếu xoá chúng.

6. Tự tạo thư mục pages/ – nơi đặt từng trang giao diện

Tạo thư mục: frontend/src/**pages**

Đây là nơi chúng ta sẽ tạo:

- ***Employees.jsx***
- ***EmployeeAdd.jsx***
- ***EmployeeEdit.jsx***

Mỗi file tương ứng 1 trang của hệ thống.

Ví dụ file:

- Employees.jsx = *hiển thị danh sách nhân viên*
- EmployeeAdd.jsx = *form thêm nhân viên*
- EmployeeEdit.jsx = *form sửa nhân viên*

Đây là thư mục QUAN TRỌNG nhất trong dự án của chúng ta.

7. Tự tạo thư mục components/ – nơi chứa các phần giao diện tái sử dụng

Tạo thư mục **components** trong thư mục **frontend/src/**

Dùng để đặt:

- *Header.jsx*
- *Sidebar.jsx*
- *Footer.jsx*
- *DashboardCard.jsx*

Ví dụ Header.jsx:

```
export default function Header() {
  return (
    <nav className="navbar navbar-dark bg-dark px-4">
      <div className="navbar-brand text-white">Dashboard</div>
```

```
    </nav>
  );
}
```

Component sẽ được import vào App.js.

8. Tự tạo thư mục services/ – để gọi API Flask bằng Axios

Tạo thư mục **services** trong thư mục **frontend/src/**

Trong thư mục services tạo file: **api.js**

Ví dụ:

```
import axios from "axios";
export const api = axios.create({
  baseURL: "http://localhost:5000/api"
});
```

Mục đích: Dùng để gọi các API, ví dụ: api.get("/employees")

BƯỚC 2 – TẠO LAYOUT DASHBOARD (BẰNG REACT + BOOTSTRAP)

Trình tự thực hiện:

1. *Cài Bootstrap*
2. *Tạo folder components/*
3. *Tạo Header.jsx*
4. *Tạo Sidebar.jsx*
5. *Tạo Layout.jsx*
6. *Kết nối Layout vào App.js*
7. *Giao diện Dashboard hoàn chỉnh xuất hiện*

Bước 2.1 Cài Bootstrap vào React

Mở **Command Prompt** di chuyển vào thư mục **frontend**, gõ lệnh: **npm install bootstrap**

```
G:\NDQUANGHUY\TAILIEUGIANGDAY\TAI LIEU DAY CMU\Software Integration Practices\CODE MAU\API_KETNOI_2DB_2026 -REACT\frontend>npm install bootstrap

added 2 packages, and audited 1317 packages in 5s

270 packages are looking for funding
  run `npm fund` for details

28 vulnerabilities (2 moderate, 26 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.

G:\NDQUANGHUY\TAILIEUGIANGDAY\TAI LIEU DAY CMU\Software Integration Practices\CODE MAU\API_KETNOI_2DB_2026 -REACT\frontend>
```

Sau đó mở file: frontend/src/**index.js**

Thêm dòng:

```
import "bootstrap/dist/css/bootstrap.min.css"; // <-- thêm dòng này
```

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';
import "bootstrap/dist/css/bootstrap.min.css"; // <-- thêm dòng này
```

Bước 2.2 Tạo folder components/

Trong thư mục: frontend/src/ tạo folder **components**

Bước 2.3 – Tạo file Header.jsx

Vị trí file: *frontend/src/components/Header.jsx*

Trong file **Header.jsx** gõ lệnh sau:

```
// Export component Header để có thể sử dụng ở các file khác (ví dụ: App.jsx)
// Đây là một React Functional Component
export default function Header() {

  // return: JSX sẽ được render ra giao diện
  return (

    // Thẻ <nav> tạo thanh điều hướng (navigation bar)
```

```
    // className là thuộc tính dùng trong React thay cho "class" của HTML
    // navbar navbar-dark bg-dark px-4 là các class của Bootstrap:
    // navbar: định dạng thanh menu
    // navbar-dark + bg-dark: nền màu đen và chữ sáng
    // px-4: padding trái phải
    <nav className="navbar navbar-dark bg-dark px-4">

      {/*
        Thẻ span hiển thị tiêu đề của navbar
        navbar-brand: style cho tên thương hiệu trong Bootstrap
        mb-0: margin-bottom = 0
        h1: kích thước chữ giống heading h1
        text-white: màu chữ trắng
      */}
      <span className="navbar-brand mb-0 h1 text-white">

        {/* Nội dung tiêu đề hiển thị trên thanh header */}
        Data Integration Dashboard

      </span>

    </nav>
  );
}
  }
```

Ghi chú:

- *Đây là thanh navbar trên cùng*
- *Luôn hiển thị ở mọi trang*

Bước 2.4 – Tạo Sidebar.jsx (Menu trái)

Vị trí file: frontend/src/components/**Sidebar.jsx**

Trong file **Sidebar.jsx** gõ lệnh sau:

```
// Import component Link từ thư viện react-router-dom
// Link dùng để điều hướng giữa các trang trong React mà không cần reload trang
import { Link } from "react-router-dom";

// Export component Sidebar để sử dụng ở các file khác (ví dụ: App.jsx)
export default function Sidebar() {

  // JSX trả về giao diện của thanh menu bên trái
```

```
  return (

    // div ngoài cùng tạo vùng sidebar
    // bg-light: nền màu sáng (Bootstrap)
    // border-end: đường viền bên phải
    // style: đặt chiều rộng sidebar = 220px và chiều cao = toàn màn hình (100vh)
    <div className="bg-light border-end" style={{ width: "220px", height: "100vh" }}>

      {/*
        list-group: component của Bootstrap để tạo danh sách menu
        list-group-flush: bỏ viền ngoài cho danh sách
      */}
      <div className="list-group list-group-flush">

        {/*
          Link điều hướng đến trang "/" (trang danh sách nhân viên)
          to="/": đường dẫn route
          list-group-item: định dạng item trong danh sách Bootstrap
          list-group-item-action: cho phép click giống button
        */}
        <Link to="/" className="list-group-item list-group-item-action">
          Employees
        </Link>

        {/*
          Link điều hướng đến trang thêm nhân viên
          Route: /employees/add
        */}
        <Link to="/employees/add" className="list-group-item list-group-item-action">
          Add Employee
        </Link>

      </div>
    </div>
  );
}
```

Ghi chú:

- *Đây là menu điều hướng*
- *Bên trái, cố định, giống Admin Dashboard*

Bước 2.5 – Tạo Layout.jsx để chứa Header + Sidebar + Nội dung

Vị trí: frontend/src/components/**Layout.jsx**

Mở file **Layout.jsx** và gõ lệnh sau:

```
// Import component Header từ file Header.jsx
// Header dùng để hiển thị thanh tiêu đề ở phía trên của website
import Header from "./Header";

// Import component Sidebar từ file Sidebar.jsx
// Sidebar là menu điều hướng ở bên trái
import Sidebar from "./Sidebar";

// Export component Layout để sử dụng trong các trang khác
// { children } là props đặc biệt của React dùng để hiển thị nội dung con bên trong Layout
export default function Layout({ children }) {

  // JSX trả về cấu trúc bố cục chung của toàn bộ ứng dụng
  return (

    // div bao ngoài cùng của layout
    <div>

      {/* Hiển thị thanh Header ở phía trên */}
      <Header />

      {/*
        div chứa Sidebar và nội dung chính
        display: flex giúp bố trí các phần tử theo chiều ngang
      */}
      <div style={{ display: "flex" }}>

        {/* Hiển thị menu bên trái */}
        <Sidebar />

        {/*
          main là vùng nội dung chính của trang
          flexGrow: 1 giúp phần này chiếm toàn bộ không gian còn lại
          p-4 là class Bootstrap tạo padding
        */}
        <main style={{ flexGrow: 1 }} className="p-4">

          {/*
            children là nội dung của từng trang sẽ được render tại đây
            Ví dụ: Employees page, Add Employee page
          */}
          {children}
```

```
        </main>

      </div>
    </div>
  );
}
```

Ghi chú:

- *<Header> luôn ở trên cùng*
- *<Sidebar> nằm trái, cố định*
- *{children} nội dung thay đổi theo từng trang React*

  (Employees, Add, Edit sẽ nằm ở đây)

Giống như "khung xương" của website.

Bước 2.6 – Kết hợp layout vào App.js

Vị trí file: frontend/src/**App.js**

Thay toàn bộ nội dung trong file bằng nội dung sau:

```
// Import các component cần thiết từ thư viện react-router-dom
// BrowserRouter: quản lý routing của ứng dụng React
// Routes: chứa danh sách các route
// Route: định nghĩa từng đường dẫn và component tương ứng
import { BrowserRouter, Routes, Route } from "react-router-dom";

// Import Layout để tạo bố cục chung (Header + Sidebar + Main Content)
import Layout from "./components/Layout";

// Import các trang (pages) của ứng dụng
// Employees: trang hiển thị danh sách nhân viên
import Employees from "./pages/Employees";

// EmployeeAdd: trang thêm nhân viên mới
import EmployeeAdd from "./pages/EmployeeAdd";

// EmployeeEdit: trang chỉnh sửa thông tin nhân viên
import EmployeeEdit from "./pages/EmployeeEdit";

// Component chính của toàn bộ ứng dụng React
function App() {
```

```
  // JSX trả về cấu trúc routing của ứng dụng
  return (

    // BrowserRouter bao toàn bộ ứng dụng để kích hoạt hệ thống routing
    <BrowserRouter>

      {/* Layout chứa Header + Sidebar + vùng nội dung */}
      <Layout>

        {/* Routes chứa tất cả các đường dẫn của ứng dụng */}
        <Routes>

          {/*
            Route trang chủ
            path="/" nghĩa là khi truy cập đường dẫn gốc
            element={<Employees />} nghĩa là render component Employees
          */}
          <Route path="/" element={<Employees />} />

          {/*
            Route thêm nhân viên
            Khi truy cập /employees/add sẽ hiển thị trang EmployeeAdd
          */}
          <Route path="/employees/add" element={<EmployeeAdd />} />

          {/*
            Route chỉnh sửa nhân viên
            :id là tham số động (dynamic parameter)
            Ví dụ: /employees/5
            Trang EmployeeEdit sẽ nhận id để load dữ liệu nhân viên tương ứng
          */}
          <Route path="/employees/:id" element={<EmployeeEdit />} />

        </Routes>

      </Layout>

    </BrowserRouter>
  );
}

// Export App để sử dụng trong file index.js
export default App;
```

Bây giờ tất cả giao diện đều chui vào Layout:

- *Header: luôn trên cùng*
- *Sidebar: luôn bên trái*
- *children}: trang Employees/Add/Edit*

Bước 2.7 – Cài react-router-dom

Trong Terminal của Visual Studio Code, Di chuyển đến thư mục frontend bằng lệnh **cd frontend**

Cài **react-router-dom** bằng lệnh **npm install react-router-dom**

Bước 2.8 – Tạo các file

Tạo thư mục **pages** trong **/src**

Trong thư mục **pages** lần lượt tạo các file sau với nội dung để trống

- TẠO file **Employees.jsx**

```
export default function Employees() {
    return <h2>Employees Page</h2>;
  }
```

- TẠO file **EmployeeAdd.jsx**

```
export default function EmployeeAdd() {
    return <h2>Add Employee Page</h2>;
  }
```

- TẠO file **EmployeeEdit.jsx**

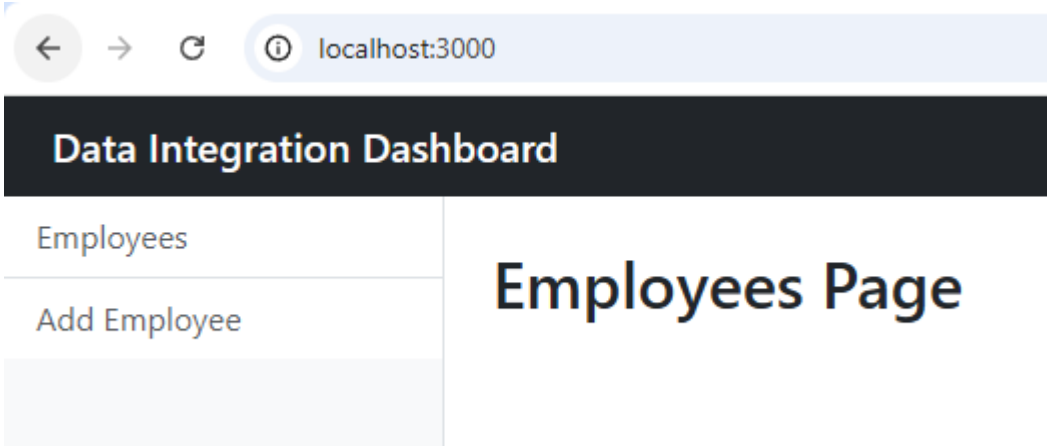
```
export default function EmployeeEdit() {
    return <h2>Edit Employee Page</h2>;
  }
```

Bước 2.9 – Chạy React để xem Dashboard

Trong terminal của Visual Studio Code, Di chuyển đến thư mục frontend bằng lệnh **cd frontend**

Sau đó, gõ lệnh: **npm start**

Nếu thành công, màn hình sẽ hiển thị như hình dưới



Chúng ta đã hoàn thành FE cơ bản của LAB với REACT

PHẦN 8: THIẾT KẾ BACK-END

Bước 1: Tạo file config.py

```
import pyodbc
import mysql.connector

# ============================================
# KẾT NỐI SQL SERVER (HUMAN)
# Hàm này tạo kết nối tới DB HUMAN bằng ODBC
# ============================================
def get_sqlserver_connection():
    try:
        conn = pyodbc.connect(
            "DRIVER={ODBC Driver 17 for SQL Server};"
            "SERVER=localhost;"       # Tên server
            "DATABASE=HUMAN;"         # Tên database Human Resource
            "UID=sa;"                 # Username SQL Server
            "PWD=123456;",            # Password
            timeout=5
        )
        return conn

    except Exception as e:
        print("Lỗi kết nối SQL Server:", str(e))
```

```
        raise


# ==========================================
# KẾT NỐI MYSQL (PAYROLL)
# Dùng mysql.connector với autocommit = False
# ==========================================
def get_mysql_connection():
    try:
        conn = mysql.connector.connect(
            host="localhost",
            user="root",
            password="123456",
            database="PAYROLL",
            autocommit=False         # REQUIRED cho 2-phase-commit
        )
        return conn

    except Exception as e:
        print("Lỗi kết nối MySQL:", str(e))
        raise
```

Bước 2: Tạo file app.py

Cài bổ sung thư viện **pip install flask-cors**

Nếu không cài thư viện này thì xãy ra lỗi: Blocked by CORS policy

Vì:

- *React chạy ở localhost:3000*
- *Flask chạy ở localhost:5000*
- *Hai domain khác nhau do đó cần phải bật CORS*

Trong thư mục project, tạo file **app.py**

```
from flask import Flask, render_template
from router import router                # Import các API
from flask_cors import CORS

app = Flask(__name__)
CORS(app)
app.register_blueprint(router)          # Đăng ký router chứa API
```

```
# ==============================
# TRANG DANH SÁCH NHÂN VIÊN
# ==============================
@app.route("/")
def index():
    return render_template("employees.html")

# ==============================
# TRANG THÊM NHÂN VIÊN
# ==============================
@app.route("/employees/add")
def employees_add_page():
    return render_template("employee_add.html")

# ==============================
# TRANG CHỈNH SỬA (TRUYỀN ID)
# ==============================
@app.route("/employees/<int:emp_id>")
def employee_edit_page(emp_id):
    return render_template("employee_edit.html", emp_id=emp_id)

if __name__ == "__main__":
    app.run(debug=True)
```

Bước 3: Tạo file router.py

```
# ============================================================
# 1. IMPORT THƯ VIỆN
# ============================================================

from flask import Blueprint, jsonify, request
# - Blueprint: giúp chia cấu trúc API thành từng nhóm riêng.
# - jsonify: dùng để trả dữ liệu JSON về cho Frontend (JavaScript).
# - request: dùng để lấy dữ liệu gửi từ Frontend lên (POST, PUT).


from config import get_sqlserver_connection, get_mysql_connection
# Import 2 hàm do bạn định nghĩa trong file config.py.
# Mỗi hàm sẽ tạo kết nối đến đúng database tương ứng:
# - get_sqlserver_connection(): tạo kết nối SQL Server (HUMAN)
# - get_mysql_connection(): tạo kết nối MySQL (PAYROLL)
```

```python
# ============================================================
# 2. TẠO ĐỐI TƯỢNG BLUEPRINT CHỨA TOÀN BỘ API
# ============================================================

router = Blueprint("router", __name__)
# "router" là tên của Blueprint.
# __name__ giúp Flask biết vị trí file để map route.

# ============================================================
# 0. API: LẤY DANH SÁCH PHÒNG BAN (DEPARTMENTS)
# ============================================================

@router.route("/api/departments")
def get_departments():

    sql = get_sqlserver_connection()
    # Mở kết nối tới database SQL Server (HUMAN)

    cur = sql.cursor()
    # cursor = con trỏ, dùng để thực thi câu lệnh SQL


    cur.execute("""
        SELECT DepartmentID, DepartmentName
        FROM Departments
        ORDER BY DepartmentName
    """)
    # Lấy toàn bộ phòng ban, sắp xếp theo tên


    rows = [
        {"DepartmentID": r[0], "DepartmentName": r[1]}
        for r in cur.fetchall()
    ]
    # fetchall() trả toàn bộ kết quả từ DB.
    # r[0] = DepartmentID, r[1] = DepartmentName
    # Chuyển thành list các object JSON.


    return jsonify(rows)
    # Trả dữ liệu JSON về Frontend.




# ============================================================
```

```
# 0. API: LẤY DANH SÁCH CHỨC VỤ (POSITIONS)
# ============================================================

@router.route("/api/positions")
#Khi client (Frontend) gửi request GET tới đường dẫn /api/positions thì gọi hàm
get_positions() để xử lý.
def get_positions(): # Định nghĩa một hàm tên là get_positions.
                     # Hàm này sẽ thực thi mỗi khi API /api/positions được gọi.

    sql = get_sqlserver_connection() #Gọi hàm get_sqlserver_connection() trong file config.py
để Kết nối tới SQL Server database
    cur = sql.cursor() #Tạo cursor, nghĩa là "con trỏ truy vấn, dùng để: Gửi lệnh SQL tới
database, Lấy dữ liệu trả về từ database
    # cur.execute() dùng để chạy câu lệnh SQL trên SQL Server.
    cur.execute("""
        SELECT PositionID, PositionName
        FROM Positions
        ORDER BY PositionName
    """)

    rows = [
        {"PositionID": r[0], "PositionName": r[1]}
        for r in cur.fetchall() # cur.fetchall(): Lấy toàn bộ kết quả trả về từ câu lệnh SQL,
Kết quả trả về là danh sách các dòng
    ]

    return jsonify(rows) #Trả rows về cho Frontend dưới dạng JSON Array




# ============================================================
# 1. API: LẤY DANH SÁCH NHÂN VIÊN (GET EMPLOYEES)
# ============================================================

@router.route("/api/employees")
def get_employees():

    sql = get_sqlserver_connection()
    cur = sql.cursor()

    # Query JOIN 3 bảng để lấy tên phòng ban & chức vụ
    cur.execute("""
        SELECT e.EmployeeID, e.FullName, d.DepartmentName, p.PositionName
        FROM Employees e
```

```
        LEFT JOIN Departments d ON e.DepartmentID = d.DepartmentID
        LEFT JOIN Positions p ON e.PositionID = p.PositionID
        ORDER BY e.EmployeeID
    """)

    rows = []  # list rỗng để bỏ từng nhân viên vào
    for r in cur.fetchall():
        # Với mỗi dòng dữ liệu trong DB, ta chuyển sang JSON object
        rows.append({
            "EmployeeID": r[0],   # ID của nhân viên
            "FullName": r[1],     # Họ tên
            "Department": r[2],   # Tên phòng ban
            "Position": r[3]      # Tên chức vụ
        })

    return jsonify(rows)
# ============================================================
# 2. API: LẤY CHI TIẾT NHÂN VIÊN THEO ID (GET EMPLOYEE DETAIL)
# ============================================================

@router.route("/api/employees/<int:emp_id>")
def get_employee_detail(emp_id):
    # emp_id là biến lấy từ URL, kiểu int

    sql = get_sqlserver_connection()
    cur = sql.cursor()

    cur.execute("""
        SELECT
            e.EmployeeID,
            e.FullName,
            e.Email,
            e.DateOfBirth,
            e.Gender,
            e.PhoneNumber,
            e.HireDate,
            e.Status,
            d.DepartmentID,
            d.DepartmentName,
            p.PositionID,
            p.PositionName
        FROM Employees e
        LEFT JOIN Departments d ON e.DepartmentID = d.DepartmentID
        LEFT JOIN Positions p ON e.PositionID = p.PositionID
        WHERE EmployeeID = ?
```

```
        """, emp_id)
    # Dấu ? là placeholder, emp_id là giá trị cần gán

    r = cur.fetchone()  # Lấy 1 dòng duy nhất

    if not r:  # Nếu không tìm thấy nhân viên
        return jsonify({"msg": "Employee not found"}), 404

    return jsonify({
        "EmployeeID": r[0],
        "FullName": r[1],
        "Email": r[2],
        "DateOfBirth": r[3],
        "Gender": r[4],
        "PhoneNumber": r[5],
        "HireDate": r[6],
        "Status": r[7],
        "DepartmentID": r[8],
        "DepartmentName": r[9],
        "PositionID": r[10],
        "PositionName": r[11]
    })

# ============================================================
# 3. API: THÊM NHÂN VIÊN MỚI (CREATE EMPLOYEE)
# ============================================================

@router.route("/api/employees", methods=["POST"])
def add_employee():

    data = request.get_json()
    # Lấy JSON mà Frontend gửi qua fetch()

    # Lấy từng trường dữ liệu
    full_name   = data.get("FullName")
    dob         = data.get("DateOfBirth")
    gender      = data.get("Gender")
    phone       = data.get("PhoneNumber")
    email       = data.get("Email")
    hire_date   = data.get("HireDate")
    dept_id     = data.get("DepartmentID") or None
    pos_id      = data.get("PositionID") or None
    status      = data.get("Status") or "Active"
```

```
    # -------------------------------------------------------
    # 1. CHECK EMAIL TRÙNG TRONG SQL SERVER
    # -------------------------------------------------------
    sql = get_sqlserver_connection()
    cur = sql.cursor()

    cur.execute("SELECT COUNT(*) FROM Employees WHERE Email = ?", email)

    if cur.fetchone()[0] > 0:
        # Nếu số lượng > 0 => email đã tồn tại
        return jsonify({"status": "error", "msg": "Email đã tồn tại"}), 400


    # -------------------------------------------------------
    # 2. BẮT ĐẦU TRANSACTION CHO 2 DATABASE
    # -------------------------------------------------------
    my = get_mysql_connection()

    sql.autocommit = False   # Buộc SQL Server chờ commit
    my.start_transaction()   # Bắt đầu transaction trong MySQL


    try:
        # -------------------------------------------------------
        # 3. INSERT SQL SERVER VÀ LẤY EMPLOYEEID MỚI
        # -------------------------------------------------------
        cur.execute("""
            INSERT INTO Employees
            (FullName, DateOfBirth, Gender, PhoneNumber, Email,
             HireDate, DepartmentID, PositionID, Status)
            OUTPUT INSERTED.EmployeeID
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            full_name, dob, gender, phone, email,
            hire_date, dept_id, pos_id, status
        ))

        row = cur.fetchone()       # Lấy EmployeeID mới tạo
        new_id = int(row[0])


        # -------------------------------------------------------
        # 4. INSERT SANG MYSQL ĐỂ ĐỒNG BỘ PAYROLL
        # -------------------------------------------------------
        my_cur = my.cursor(dictionary=True)
```

```
        my_cur.execute("""
            INSERT INTO employees_payroll
            (EmployeeID, FullName, DepartmentID, PositionID, Status)
            VALUES (%s, %s, %s, %s, %s)
        """, (
            new_id, full_name, dept_id, pos_id, status
        ))

        # Cả 2 đều thành công → commit 2 DB
        sql.commit()
        my.commit()


    except Exception as e:
        # Nếu có lỗi ở 1 trong 2 DB → rollback cả 2
        sql.rollback()
        my.rollback()

        return jsonify({"status": "error", "msg": str(e)}), 500


    return jsonify({
        "status": "success",
        "msg": f"Thêm nhân viên thành công (ID = {new_id})"
    })


# ============================================================
# 4. API: UPDATE NHÂN VIÊN
# ============================================================

@router.route("/api/employees/<int:emp_id>", methods=["PUT"])
def update_employee(emp_id):

    data = request.get_json()

    # Lấy dữ liệu từ React
    full_name = data.get("FullName")
    dob       = data.get("DateOfBirth")
    gender    = data.get("Gender")
    phone     = data.get("PhoneNumber")
    email     = data.get("Email")
    hire_date = data.get("HireDate")
    dept_id   = data.get("DepartmentID")
```

```
    pos_id    = data.get("PositionID")
    status    = data.get("Status")

    sql = get_sqlserver_connection()
    my  = get_mysql_connection()

    sql.autocommit = False
    my.start_transaction()

    try:

        # ============================================
        # UPDATE SQL SERVER
        # ============================================
        cur = sql.cursor()

        cur.execute("""
            UPDATE Employees
            SET
                FullName=?,
                DateOfBirth=?,
                Gender=?,
                PhoneNumber=?,
                Email=?,
                HireDate=?,
                DepartmentID=?,
                PositionID=?,
                Status=?
            WHERE EmployeeID=?
        """, (
            full_name,
            dob,
            gender,
            phone,
            email,
            hire_date,
            dept_id,
            pos_id,
            status,
            emp_id
        ))

        # ============================================
        # UPDATE MYSQL (PAYROLL)
        # ============================================
```

```
        my_cur = my.cursor(dictionary=True)

        my_cur.execute("""
            UPDATE employees_payroll
            SET
                FullName=%s,
                DepartmentID=%s,
                PositionID=%s,
                Status=%s
            WHERE EmployeeID=%s
        """, (
            full_name,
            dept_id,
            pos_id,
            status,
            emp_id
        ))

        sql.commit()
        my.commit()

    except Exception as e:

        sql.rollback()
        my.rollback()

        return jsonify({
            "status": "error",
            "msg": str(e)
        }), 500

    return jsonify({
        "status": "success",
        "msg": "Update thành công"
    })
# ============================================================
# 5. API: XÓA NHÂN VIÊN
# ============================================================

@router.route("/api/employees/<int:emp_id>", methods=["DELETE"])
def delete_employee(emp_id):

    sql = get_sqlserver_connection()
    my  = get_mysql_connection()
```

```
    sql.autocommit = False
    my.start_transaction()

    try:
        cur = sql.cursor()

        # ==================================================
        # CHECK RÀNG BUỘC: NẾU NHÂN VIÊN CÓ DIVIDENDS → KHÔNG XOÁ
        # ==================================================
        cur.execute("SELECT COUNT(*) FROM Dividends WHERE EmployeeID=?", emp_id)

        if cur.fetchone()[0] > 0:
            return jsonify({
                "status": "error",
                "msg": "Không thể xoá – nhân viên có Dividends"
            }), 400


        # ==================================================
        # XÓA TRONG SQL SERVER
        # ==================================================
        cur.execute("DELETE FROM Employees WHERE EmployeeID=?", emp_id)


        # ==================================================
        # XÓA TRONG MYSQL (PAYROLL, ATTENDANCE, SALARIES)
        # ==================================================
        my_cur = my.cursor(dictionary=True)

        my_cur.execute("DELETE FROM employees_payroll WHERE EmployeeID=%s", (emp_id,))
        my_cur.execute("DELETE FROM attendance WHERE EmployeeID=%s", (emp_id,))
        my_cur.execute("DELETE FROM salaries WHERE EmployeeID=%s", (emp_id,))



        sql.commit()
        my.commit()
    except Exception as e:
        sql.rollback()
        my.rollback()
        return jsonify({"status": "error", "msg": str(e)}), 500


    return jsonify({"status": "success", "msg": "Xoá thành công"})
```

PHẦN 9: THIẾT KẾ FRONT-END CHO CÁC CHỨC NĂNG CỦA DỰ ÁN

Bước 1: In danh sách nhân viên

Mở file **Employees.jsx** trong thư mục / **pages**

Gõ nội dung sau:

```
// Import các hook của React
// useState: dùng để lưu dữ liệu trong component
// useEffect: dùng để thực hiện hành động khi component được render (ví dụ gọi API)
import { useEffect, useState } from "react";

// Import Link từ react-router-dom để chuyển trang mà không reload trang
import { Link } from "react-router-dom";

// Component Employees dùng để hiển thị danh sách nhân viên
export default function Employees() {

  // Khai báo state employees để lưu danh sách nhân viên
  // useState([]) nghĩa là giá trị ban đầu là mảng rỗng
  const [employees, setEmployees] = useState([]);

  // Hàm gọi API từ backend Flask để lấy danh sách nhân viên
  const loadEmployees = () => {

    // fetch dùng để gọi HTTP request tới API backend
    fetch("http://localhost:5000/api/employees")

      // Chuyển dữ liệu response thành JSON
      .then((res) => res.json())

      // Sau khi nhận dữ liệu từ backend
      .then((data) => {

        // Lưu dữ liệu vào state employees
        // Khi state thay đổi React sẽ tự động render lại giao diện
        setEmployees(data);
      })

      // Bắt lỗi nếu API bị lỗi
      .catch((err) => console.error("Lỗi tải danh sách:", err));
  };

  // useEffect được gọi khi component được render lần đầu
  // [] nghĩa là chỉ chạy 1 lần khi trang load
```

```
useEffect(() => {
  loadEmployees();
}, []);

// Hàm xóa nhân viên
const deleteEmployee = (id) => {

  // Hiển thị hộp thoại xác nhận trước khi xóa
  if (!window.confirm("Bạn chắc chắn muốn xóa nhân viên này?")) return;

  // Gửi request DELETE tới API Flask
  fetch(`http://localhost:5000/api/employees/${id}`, {
    method: "DELETE",
  })

    // Nhận kết quả trả về từ server
    .then((res) => res.json())

    .then((rs) => {

      // Hiển thị thông báo từ backend
      alert(rs.msg);

      // Nếu xóa thành công thì tải lại danh sách nhân viên
      if (rs.status === "success") loadEmployees();
    });
};

// JSX trả về giao diện trang Employees
return (
  <div>

    {/* Tiêu đề trang */}
    <h3>Employee List</h3>

    {/* Nút chuyển sang trang thêm nhân viên */}
    <Link to="/employees/add" className="btn btn-success mb-3">
      + Add Employee
    </Link>

    {/* Bảng hiển thị danh sách nhân viên */}
    <table className="table table-bordered">

      {/* Phần tiêu đề của bảng */}
      <thead className="table-dark">
```

```
          <tr>
            <th>ID</th>
            <th>Full Name</th>
            <th>Department</th>
            <th>Position</th>
            <th style={{ width: "150px" }}>Action</th>
          </tr>
        </thead>

        {/* Phần dữ liệu của bảng */}
        <tbody>

          {/* Dùng map() để lặp qua danh sách employees */}
          {employees.map((emp) => (

            // key giúp React quản lý danh sách hiệu quả
            <tr key={emp.EmployeeID}>

              {/* Hiển thị thông tin từng nhân viên */}
              <td>{emp.EmployeeID}</td>
              <td>{emp.FullName}</td>
              <td>{emp.Department}</td>
              <td>{emp.Position}</td>

              <td>

                {/* Nút Edit để chuyển sang trang chỉnh sửa */}
                <Link
                  className="btn btn-primary btn-sm me-2"
                  to={`/employees/${emp.EmployeeID}`}
                >
                  Edit
                </Link>

                {/* Nút Delete để xóa nhân viên */}
                <button
                  className="btn btn-danger btn-sm"
                  onClick={() => deleteEmployee(emp.EmployeeID)}
                >
                  Delete
                </button>

              </td>
            </tr>
          ))}
```

```
          {/* Trường hợp danh sách rỗng */}
          {employees.length === 0 && (
            <tr>
              <td colSpan="5" className="text-center text-muted">
                No data available.
              </td>
            </tr>
          )}

        </tbody>
      </table>
    </div>
  );
}
```

Bước 2: Thêm mới hồ sơ nhân viên

Code sẽ:

- *Hiển thị form nhập FullName, DOB, Gender, Phone, Email, HireDate*
- *Tự động tải danh sách **Department** từ API /api/departments*
- *Tự động tải danh sách **Position** từ API /api/positions*
- *Gửi yêu cầu **POST** tới API Flask /api/employees*
- *Hiển thị thông báo thành công*
- *Điều hướng quay lại danh sách nhân viên*

Mở file **EmployeeAdd.jsx** trong thư mục **/pages**

Gõ nội dung sau:

```
// Import các React hooks
// useState: dùng để lưu dữ liệu trong component
// useEffect: dùng để chạy code khi component được render
import { useEffect, useState } from "react";

// useNavigate dùng để chuyển trang bằng code (programmatic navigation)
import { useNavigate } from "react-router-dom";

// Component dùng để thêm nhân viên mới
export default function EmployeeAdd() {

  // nav dùng để điều hướng trang
```

```
const nav = useNavigate();

// STATE lưu dữ liệu của form
// Mỗi field tương ứng với một cột trong database
const [form, setForm] = useState({
  FullName: "",
  DateOfBirth: "",
  Gender: "",
  PhoneNumber: "",
  Email: "",
  HireDate: "",
  DepartmentID: "",
  PositionID: "",
  Status: "Active", // giá trị mặc định
});

// STATE lưu dữ liệu cho dropdown
const [departments, setDepartments] = useState([]);
const [positions, setPositions] = useState([]);

// Hàm xử lý khi người dùng thay đổi dữ liệu trong form
const handleChange = (e) => {

  // Cập nhật state form
  setForm({
    ...form,                  // giữ nguyên các field cũ
    [e.target.id]: e.target.value, // cập nhật field đang thay đổi
  });
};

// Hàm load dữ liệu cho dropdown (department và position)
const loadDropdowns = () => {

  // Gọi API lấy danh sách phòng ban
  fetch("http://localhost:5000/api/departments")
    .then((r) => r.json())
    .then((data) => setDepartments(data));

  // Gọi API lấy danh sách chức vụ
  fetch("http://localhost:5000/api/positions")
    .then((r) => r.json())
    .then((data) => setPositions(data));
};

// Hàm gửi dữ liệu form lên backend Flask
```

```
const handleSubmit = (e) => {

  // Ngăn form reload trang
  e.preventDefault();

  // Gửi request POST đến API
  fetch("http://localhost:5000/api/employees", {
    method: "POST",

    // Header báo server biết dữ liệu gửi là JSON
    headers: { "Content-Type": "application/json" },

    // Chuyển object form thành JSON
    body: JSON.stringify(form),
  })

    // Nhận kết quả từ server
    .then((r) => r.json())

    .then((res) => {

      // Hiển thị thông báo từ backend
      alert(res.msg);

      // Nếu thêm thành công thì quay lại trang danh sách nhân viên
      if (res.status === "success") {
        nav("/");
      }
    });
};

// useEffect chạy khi component được mở
// Dùng để load dữ liệu dropdown
useEffect(() => {
  loadDropdowns();
}, []);

// JSX trả về giao diện của form
return (
  <div>

    {/* Tiêu đề trang */}
    <h3>Add New Employee</h3>

    {/* Form nhập thông tin nhân viên */}
```

```
      <form onSubmit={handleSubmit} className="card p-4 mt-3">

        {/* Full Name */}
        <label>Full Name</label>
        <input
          id="FullName"
          className="form-control mb-2"
          value={form.FullName}
          onChange={handleChange}
          required
        />

        {/* Date of Birth */}
        <label>Date of Birth</label>
        <input
          type="date"
          id="DateOfBirth"
          className="form-control mb-2"
          value={form.DateOfBirth}
          onChange={handleChange}
          required
        />

        {/* Gender */}
        <label>Gender</label>
        <select
          id="Gender"
          className="form-control mb-2"
          value={form.Gender}
          onChange={handleChange}
          required
        >
          <option value="">-- Select Gender --</option>
          <option>Nam</option>
          <option>Nữ</option>
          <option>Khác</option>
        </select>

        {/* Phone Number */}
        <label>Phone Number</label>
        <input
          id="PhoneNumber"
          className="form-control mb-2"
          value={form.PhoneNumber}
          onChange={handleChange}
```

```
          required
        />

        {/* Email */}
        <label>Email</label>
        <input
          id="Email"
          className="form-control mb-2"
          value={form.Email}
          onChange={handleChange}
          required
        />

        {/* Hire Date */}
        <label>Hire Date</label>
        <input
          type="date"
          id="HireDate"
          className="form-control mb-2"
          value={form.HireDate}
          onChange={handleChange}
          required
        />

        {/* Department Dropdown */}
        <label>Department</label>
        <select
          id="DepartmentID"
          className="form-control mb-2"
          value={form.DepartmentID}
          onChange={handleChange}
        >
          <option value="">-- Select Department --</option>

          {/* Lặp qua danh sách departments để tạo option */}
          {departments.map((d) => (
            <option key={d.DepartmentID} value={d.DepartmentID}>
              {d.DepartmentName}
            </option>
          ))}
        </select>

        {/* Position Dropdown */}
        <label>Position</label>
        <select
```

```
          id="PositionID"
          className="form-control mb-2"
          value={form.PositionID}
          onChange={handleChange}
        >
          <option value="">-- Select Position --</option>

          {/* Lặp qua danh sách positions */}
          {positions.map((p) => (
            <option key={p.PositionID} value={p.PositionID}>
              {p.PositionName}
            </option>
          ))}
        </select>

        {/* Status */}
        <label>Status</label>
        <select
          id="Status"
          className="form-control mb-2"
          value={form.Status}
          onChange={handleChange}
        >
          <option value="Active">Active</option>
          <option value="Inactive">Inactive</option>
        </select>

        {/* Nút submit form */}
        <button className="btn btn-primary mt-2">
          Add Employee
        </button>

      </form>
    </div>
  );
}
```

Bước 3: Cập nhật thông tin hồ sơ nhân viên

Code sẽ:

- *Lấy đúng **ID nhân viên** từ URL*
- *Gọi API Flask:*

  - *GET /api/employees/<id> → hiển thị dữ liệu lên form*
  - *PUT /api/employees/<id> → lưu lại chỉnh sửa*
- *Có dropdown Department và Position*
- *Có nút "Save changes"*
- *Sau khi lưu quay lại trang danh sách nhân viên*

Mở file **EmployeeEdit.jsx** trong thư mục **/pages**

Gõ nội dung sau:

```
// Import các React hooks
// useState: dùng để lưu dữ liệu form và dropdown
// useEffect: dùng để chạy code khi component được render
import { useEffect, useState } from "react";

// useNavigate: chuyển trang bằng code
// useParams: lấy tham số từ URL (ví dụ /employees/5 → id = 5)
import { useNavigate, useParams } from "react-router-dom";

// Component dùng để chỉnh sửa thông tin nhân viên
export default function EmployeeEdit() {

  // Hook điều hướng trang
  const nav = useNavigate();

  // Lấy id từ URL
  // Ví dụ URL: /employees/5 → id = 5
  const { id } = useParams();

  // STATE lưu dữ liệu form nhân viên
  const [form, setForm] = useState({
    FullName: "",
    DateOfBirth: "",
    Gender: "",
    PhoneNumber: "",
    Email: "",
    HireDate: "",
    DepartmentID: "",
    PositionID: "",
    Status: "",
  });

  // STATE lưu dữ liệu dropdown
  const [departments, setDepartments] = useState([]);
  const [positions, setPositions] = useState([]);
```

```
// Hàm xử lý khi người dùng thay đổi dữ liệu input
const handleChange = (e) => {

  // Cập nhật state form
  setForm({
    ...form, // giữ nguyên dữ liệu cũ
    [e.target.id]: e.target.value, // cập nhật field vừa thay đổi
  });
};

// Hàm chuyển định dạng ngày
// Backend thường trả dạng GMT
// Input HTML date cần dạng YYYY-MM-DD
const convertDate = (dt) => {
  if (!dt) return "";

  return new Date(dt)
    .toISOString()
    .substring(0, 10);
};

// Hàm load dữ liệu dropdown
const loadDropdowns = () => {

  // Gọi API lấy danh sách phòng ban
  fetch("http://localhost:5000/api/departments")
    .then((res) => res.json())
    .then((data) => setDepartments(data))
    .catch(() => alert("Không load được danh sách phòng ban"));

  // Gọi API lấy danh sách chức vụ
  fetch("http://localhost:5000/api/positions")
    .then((res) => res.json())
    .then((data) => setPositions(data))
    .catch(() => alert("Không load được danh sách chức vụ"));
};

// Hàm load dữ liệu chi tiết của nhân viên
const loadEmployee = () => {

  // Gọi API lấy thông tin nhân viên theo ID
  fetch(`http://localhost:5000/api/employees/${id}`)

    .then((res) => res.json())
```

```
    .then((data) => {

      // Đưa dữ liệu từ API vào form
      setForm({
        FullName: data.FullName || "",
        DateOfBirth: convertDate(data.DateOfBirth),
        Gender: data.Gender || "",
        PhoneNumber: data.PhoneNumber || "",
        Email: data.Email || "",
        HireDate: convertDate(data.HireDate),
        DepartmentID: data.DepartmentID || "",
        PositionID: data.PositionID || "",
        Status: data.Status || "Active",
      });
    })

    .catch(() => alert("Không tải được dữ liệu nhân viên!"));
};

// Hàm gửi dữ liệu cập nhật lên backend
const handleSubmit = (e) => {

  // Ngăn form reload trang
  e.preventDefault();

  // Gửi request PUT tới API
  fetch(`http://localhost:5000/api/employees/${id}`, {
    method: "PUT",

    // Header báo server dữ liệu gửi là JSON
    headers: { "Content-Type": "application/json" },

    // Chuyển object form thành JSON
    body: JSON.stringify(form),
  })

    .then((res) => res.json())

    .then((rs) => {

      // Hiển thị thông báo từ backend
      alert(rs.msg);

      // Nếu cập nhật thành công → quay về danh sách nhân viên
```

```
      if (rs.status === "success") {
        nav("/");
      }
    })

    .catch(() => alert("Không thể cập nhật nhân viên"));
};

// useEffect chạy khi trang được load
useEffect(() => {

  // Load dropdown
  loadDropdowns();

  // Load dữ liệu nhân viên
  loadEmployee();

}, []);

// JSX giao diện chỉnh sửa nhân viên
return (
  <div>

    {/* Tiêu đề trang */}
    <h3>Edit Employee</h3>

    {/* Form chỉnh sửa */}
    <form onSubmit={handleSubmit} className="card p-4 mt-3">

      {/* Full Name */}
      <label>Full Name</label>
      <input
        id="FullName"
        className="form-control mb-2"
        value={form.FullName}
        onChange={handleChange}
        required
      />

      {/* Date of Birth */}
      <label>Date of Birth</label>
      <input
        type="date"
        id="DateOfBirth"
        className="form-control mb-2"
```

```
          value={form.DateOfBirth}
          onChange={handleChange}
          required
        />

        {/* Gender */}
        <label>Gender</label>
        <select
          id="Gender"
          className="form-control mb-2"
          value={form.Gender}
          onChange={handleChange}
          required
        >
          <option value="">-- Select Gender --</option>
          <option>Nam</option>
          <option>Nữ</option>
          <option>Khác</option>
        </select>

        {/* Phone Number */}
        <label>Phone Number</label>
        <input
          id="PhoneNumber"
          className="form-control mb-2"
          value={form.PhoneNumber}
          onChange={handleChange}
          required
        />

        {/* Email */}
        <label>Email</label>
        <input
          id="Email"
          type="email"
          className="form-control mb-2"
          value={form.Email}
          onChange={handleChange}
          required
        />

        {/* Hire Date */}
        <label>Hire Date</label>
        <input
          type="date"
```

```
          id="HireDate"
          className="form-control mb-2"
          value={form.HireDate}
          onChange={handleChange}
          required
        />

        {/* Department */}
        <label>Department</label>
        <select
          id="DepartmentID"
          className="form-control mb-2"
          value={form.DepartmentID}
          onChange={handleChange}
          required
        >
          <option value="">-- Select Department --</option>

          {/* Tạo danh sách phòng ban */}
          {departments.map((d) => (
            <option key={d.DepartmentID} value={d.DepartmentID}>
              {d.DepartmentName}
            </option>
          ))}
        </select>

        {/* Position */}
        <label>Position</label>
        <select
          id="PositionID"
          className="form-control mb-2"
          value={form.PositionID}
          onChange={handleChange}
          required
        >
          <option value="">-- Select Position --</option>

          {/* Tạo danh sách chức vụ */}
          {positions.map((p) => (
            <option key={p.PositionID} value={p.PositionID}>
              {p.PositionName}
            </option>
          ))}
        </select>
```

```
        {/* Status */}
        <label>Status</label>
        <select
          id="Status"
          className="form-control mb-2"
          value={form.Status}
          onChange={handleChange}
          required
        >
          <option value="Active">Active</option>
          <option value="Inactive">Inactive</option>
          <option value="Đang làm việc">Đang làm việc</option>
        </select>

        {/* Nút lưu thay đổi */}
        <button className="btn btn-primary mt-2">
          Save Changes
        </button>

      </form>
    </div>
  );
}
```

PHẦN 10 – HƯỚNG DẪN CHẠY TOÀN BỘ HỆ THỐNG

Sau khi hoàn thành toàn bộ code **Front-end React** và **Back-end Flask API**, chúng ta cần chạy hệ thống để kiểm tra các chức năng của ứng dụng.

Trong dự án này có **2 chương trình phải chạy đồng thời**:

Thành phần	Công nghệ	Port
Backend API	Python Flask	5000
Frontend	React	3000

BƯỚC 1 – CHẠY BACKEND FLASK

1.1 Mở Terminal

Mở **Terminal hoặc Command Prompt** trong thư mục project.

1.2 Chạy Flask

Gõ lệnh: **python app.py**

Nếu chạy thành công, màn hình sẽ hiển thị: ** Running on http://127.0.0.1:5000*

Điều này có nghĩa: Flask API đang chạy tại port 5000

1.3 Kiểm tra API

Mở trình duyệt và truy cập: http://localhost:5000/api/employees

Nếu API hoạt động đúng, trình duyệt sẽ hiển thị dữ liệu JSON:

```
[
 {
  "EmployeeID":1,
  "FullName":"Nguyễn Văn A",
  "Department":"IT",
  "Position":"Developer"
 },
 ...
]
```

Nếu thấy dữ liệu JSON nghĩa là **Backend đã hoạt động bình thường**.

BƯỚC 2 – CHẠY FRONTEND REACT

Sau khi Backend đã chạy, tiếp tục chạy **React**.

2.1 Mở Terminal thứ hai

Không tắt terminal Flask.

Mở **Terminal mới** và di chuyển vào thư mục frontend: **cd frontend**

2.2 Cài thư viện React (nếu chưa cài)

Chạy lệnh: **npm install**

Lệnh này sẽ cài toàn bộ thư viện được khai báo trong file **package.json.**

2.3 Chạy React

Gõ lệnh: **npm start**

Sau vài giây React sẽ khởi động và mở trình duyệt tại địa chỉ: http://localhost:3000